CONSOLIDATED REPLIT PROMPT PACKAGE

AI-Driven Team & Workplace Platform — 6 Prompts, All Features, Minimum Credits

HOW TO USE THIS DOCUMENT

- There are **6 prompts** total. Send them **in order** (Prompt 1 → 2 → 3 → 4 → 5 → 6).
- **Mode for ALL prompts:** Select AUTONOMOUS MODE → Click ECONOMY. App Testing: OFF. Code Optimization: OFF.
- **Test after each prompt** before sending the next one.
- Each prompt builds the full UI + interactions + database in a single pass to save credits.
- **Do NOT re-send a prompt if something breaks.** Describe the issue to Claude and get a small fix prompt instead.

PROMPT 1 OF 6 — FOUNDATION: WORKSPACE, TEAMS, MEMBERS & PERMISSIONS

🔵 **REPLIT SETTINGS: AUTONOMOUS MODE → ECONOMY | App Testing: OFF | Code Optimization: OFF**

(Copy everything below this line into Replit)

CONTEXT:
App: AI-driven team and workplace productivity platform.
Feature being built: Workspace creation, team management, org chart, member invites, roles, and permissions.
Current state: App has individual productivity features. No team features exist yet.
Goal: Build complete UI + interactions + database for workspaces, teams, members, and role-based access in one pass.

Files to create:
src/pages/WorkspaceSetup.tsx
src/pages/TeamManagement.tsx

src/components/teams/TeamList.tsx
src/components/teams/CreateTeamModal.tsx
src/components/teams/OrgChart.tsx
src/components/teams/TeamDetailPage.tsx
src/pages/Members.tsx
src/components/members/InviteMemberModal.tsx
src/components/members/MemberCard.tsx
src/components/members/RoleSelector.tsx

TASK:
Build the complete workspace, teams, members, and permissions system — UI, all interactions, form validation, and database persistence — in one pass. Every component must be fully functional and save/load data from the database.

IMPLEMENTATION DETAILS:

=== PART A: WORKSPACE SETUP ===

1. WORKSPACE SETUP PAGE (WorkspaceSetup.tsx):
- Full page with centered card (max-width 600px)
- Text input labeled "Workspace Name" with placeholder "e.g. Acme Corp"
- Text input labeled "Industry" with placeholder "e.g. Technology"
- Dropdown labeled "Company Size" with options: "1-10", "11-50", "51-200", "201-500", "500+"
- Button "Create Workspace" (solid blue #2563EB, full width)
- Form logic: All inputs store values in React state. On submit, validate all fields are filled — show red border (#DC2626) on empty fields with "This field is required" below each. On valid submission: insert row into workspaces table, store workspace_id in app context, show green toast "Workspace created" (top-right, auto-dismiss 3s, slide-in from right).
- On error: Show red toast "Failed to create workspace. Please try again."

2. DATABASE — WORKSPACES TABLE:
- Create table "workspaces": id (UUID, PK), name (text, not null), industry (text), company_size (text), created_by (UUID, FK to users), created_at (timestamp, default now), updated_at (timestamp, default now)

=== PART B: TEAMS ===

3. TEAM LIST SIDEBAR (TeamList.tsx):
- Vertical list section in left sidebar
- Section header "Teams" with "+" icon button on right
- On mount: load all teams from teams table where workspace_id matches current workspace, order by created_at ascending
- Display each team name on left, member_count badge on right
- Clicking "+" opens CreateTeamModal

- Clicking a team row highlights it with blue (#2563EB) left border and light blue background. Only one team selected at a time. Selected team ID stored in React state.
- Empty state: "No teams yet. Click + to create your first team."

4. CREATE TEAM MODAL (CreateTeamModal.tsx):
- Centered modal overlay (max-width 480px), white background, dark overlay (rgba(0,0,0,0.4))
- Title "Create New Team"
- Text input "Team Name" placeholder "e.g. Backend Team"
- Dropdown "Team Type" options: "Permanent Department", "Project Team", "Cross-Functional Squad", "Virtual Team"
- Dropdown "Department" options: "Engineering", "Design", "Marketing", "Sales", "Operations", "HR", "Finance", "Custom"
- Textarea "Description" placeholder "What does this team do?"
- Buttons: "Cancel" (outline #CBD5E1 border), "Create Team" (solid blue #2563EB)
- Modal opens from TeamList "+" button. Closes on "Cancel", overlay click, or Escape key.
- Validate: Team Name required — red border if empty with "Team name is required".
- On valid submit: Insert into teams table with form values and current workspace_id. Refresh team list. Close modal. Show green toast "Team created".
- On error: Show red toast "Failed to create team. Please try again."

5. ORG CHART (OrgChart.tsx):
- Full width container with tree-style layout
- On mount: Load all teams with parent_team_id relationships from DB. Build tree structure.
- Root node = workspace name. Children = teams where parent_team_id is null. Nested children = teams where parent_team_id matches a parent team id.
- Each node card: Rounded rectangle, white background, subtle shadow (0 1px 3px rgba(0,0,0,0.1)), name bold, role below
- Connecting lines: Thin gray lines from parent to children
- Clicking a node toggles children visibility. Collapsed = small "+" on bottom edge. Expanded = small "-". Root expanded by default, others collapsed. Animate with 200ms ease.
- Empty state: Workspace name as single root with "Add teams to build your org chart"

6. TEAM DETAIL PAGE (TeamDetailPage.tsx inside TeamManagement.tsx):
- Full page: Header with team name (large, bold), team type badge, member count, description
- 3 tabs: "Members" (default active), "Projects", "Analytics"
- Clicking tabs switches active tab. Active = blue bottom border + blue text. Inactive = gray.
- Members tab: "No members added yet" | Projects tab: "No projects linked yet" | Analytics tab: "Analytics will appear once the team has activity"

7. DATABASE — TEAMS TABLE:
- Create table "teams": id (UUID, PK), workspace_id (UUID, FK to workspaces), name (text, not null), team_type (text, not null), department (text), description (text), parent_team_id (UUID, nullable, self-referencing FK), created_by (UUID, FK to users), created_at (timestamp, default now), member_count (integer, default 0)

=== PART C: MEMBERS & PERMISSIONS ===

8. MEMBERS PAGE (Members.tsx):
- Page header "Team Members" with subtitle showing member count (from DB row count)
- Search input (placeholder "Search members...", height 40px, border-radius 6px, search icon inside) on left, "Invite Member" button (solid blue) on right
- Grid of member cards (3 columns desktop, 2 tablet, 1 mobile, gap 16px)
- On mount: Load all rows from workspace_members where workspace_id matches. Order by display_name asc.
- Search filters displayed cards in real time by name and email (case-insensitive). Show "4 of 12 members" count.
- Empty state: "No team members yet. Invite your first member."
- Search no results: "No members found matching your search"

9. INVITE MEMBER MODAL (InviteMemberModal.tsx):
- Centered modal (max-width 480px), Title "Invite Team Member"
- Email input "Email Address" placeholder "name@company.com" — validate email format on blur, red border + "Enter a valid email" if invalid
- Role dropdown (uses RoleSelector)
- Team dropdown "Assign to Team" placeholder "Select team" — populated from teams in DB
- Buttons: "Cancel" (outline), "Send Invite" (solid blue)
- Opens from "Invite Member" button. Closes on Cancel, overlay click, Escape.
- On valid submit: Insert into workspace_members with email, role, team_id, workspace_id, invited_by=current user, status="invited". Refresh member list. Close modal. Show green toast "Invite sent".
- On error: Show red toast "Failed to send invite"

10. MEMBER CARD (MemberCard.tsx):
- White card, subtle shadow, rounded 8px, padding 16px
- Circle avatar (gray, 48px, initials), Name (bold, 16px), email (gray, 14px)
- Role badge (small rounded pill): Admin=blue, Manager=purple, Member=green, Guest=gray, Observer=orange
- Team name below (14px, gray)
- Three-dot menu icon top-right. Clicking opens dropdown: "Change Role", "Move to Team", "Remove Member". Click outside closes.
- "Change Role": Opens RoleSelector inline. On change: Update role column in workspace_members. Refresh card.
- "Remove Member": Confirmation "Remove [name]?" with Cancel and Remove buttons. On confirm: Delete row from workspace_members. Refresh list.

11. ROLE SELECTOR (RoleSelector.tsx):
- Dropdown with role options, each showing name (bold) and one-line description:
  "Owner" — "Full control over workspace and billing"

"Admin" — "Manage teams, projects, and members"
"Manager" — "Manage assigned teams and view team analytics"
"Member" — "Work on assigned tasks and projects"
"Guest" — "View specific shared projects only"
"Observer" — "Read-only access, no editing"
- Selecting: highlight row blue, close dropdown, display role name in trigger

12. DATABASE — WORKSPACE_MEMBERS TABLE:
- Create table "workspace_members": id (UUID, PK), workspace_id (UUID, FK to workspaces), user_id (UUID, FK to users, nullable — null until invite accepted), email (text, not null), display_name (text), role (text, not null, default "Member"), team_id (UUID, FK to teams, nullable), status (text, default "invited"), invited_by (UUID, FK to users), invited_at (timestamp, default now), joined_at (timestamp, nullable)

=== DESIGN REQUIREMENTS ===
- All components use existing app color scheme and font system
- Cards: rounded corners 8px, shadow 0 1px 3px rgba(0,0,0,0.1)
- Modal dark overlay: rgba(0,0,0,0.4)
- All inputs: consistent height 40px, border-radius 6px, border color #CBD5E1
- Buttons: Primary solid blue (#2563EB, white text), Secondary outline (#CBD5E1 border)
- Toast: Small rectangle top-right, green for success, red for error, auto-dismiss 3s, slide-in from right
- Selected states: blue (#2563EB) borders/highlights
- Validation errors: red (#DC2626) border and text
- All transitions: 200ms ease

=== BOUNDARIES — CRITICAL ===
- Build everything described above: UI + interactions + database
- Do not modify any existing files or features not mentioned here
- Do not add any features not described above
- Do not add any features from future modules

PROMPT 2 OF 6 — PROJECTS & TASKS CORE

🔵 **REPLIT SETTINGS: AUTONOMOUS MODE → ECONOMY | App Testing: OFF | Code Optimization: OFF**

(Copy everything below this line into Replit)

CONTEXT:
App: AI-driven team and workplace productivity platform.

Feature being built: Complete project system and task system with all properties, subtasks, and dependencies.
Current state: Workspaces, Teams, Members, and Permissions are fully working from Prompt 1.
Goal: Build complete UI + interactions + database for projects and tasks in one pass.

Files to create:
src/pages/Projects.tsx
src/components/projects/CreateProjectModal.tsx
src/components/projects/ProjectSettings.tsx
src/components/projects/ProjectHeader.tsx
src/components/tasks/TaskList.tsx
src/components/tasks/TaskDetailPanel.tsx
src/components/tasks/CreateTaskModal.tsx
src/components/tasks/SubtaskList.tsx
src/components/tasks/TaskProperties.tsx

TASK:
Build the complete project system (project list, creation, settings, navigation) and the complete task system (task CRUD, all properties, subtasks, dependencies) — UI, interactions, and database — in one pass.

IMPLEMENTATION DETAILS:

=== PART A: PROJECTS ===

1. PROJECT LIST PAGE (Projects.tsx):
- Page header "Projects" with subtitle "Manage all your team projects"
- Row: search input (placeholder "Search projects...", filters by name, case-insensitive, real-time), filter dropdown (options: "All statuses", "Active", "Planning", "On Hold", "Completed"), "New Project" button (solid blue)
- Table columns: Project Name, Status, Team, Progress, Due Date, Owner
- On mount: Load all projects from projects table where workspace_id matches, order by created_at desc. Search and filter operate client-side on loaded data.
- Clicking column header sorts by that column (toggle asc/desc, arrow icon shows direction)
- Clicking a project row navigates to that project view
- Status as colored pill badge: Active=green, Planning=blue, On Hold=orange, Completed=gray
- Progress as percentage with thin progress bar (height 4px, rounded, green fill on gray)
- Empty state: "No projects yet. Create your first project to get started."
- Table rows: Height 48px, hover light blue background

2. CREATE PROJECT MODAL (CreateProjectModal.tsx):
- Modal (max-width 560px), Title "Create New Project"
- Fields: Project Name (placeholder "e.g. Q4 Marketing Campaign" — required, red border if empty), Description textarea (placeholder "Describe the project goals..."), Team dropdown

(populated from teams DB), Start Date picker, Due Date picker, Priority dropdown ("Urgent", "High", "Medium", "Low"), Template dropdown ("Start from scratch", "Software Sprint", "Marketing Campaign", "Product Launch", "Client Project")
- Buttons: "Cancel" (outline), "Create Project" (solid blue)
- Opens from "New Project" button. Closes on Cancel, overlay click, Escape.
- On valid submit: Insert into projects table. Refresh project list and sidebar. Close modal. Show green toast "Project created".

3. PROJECT SETTINGS (ProjectSettings.tsx):
- Right-side slide-out panel (width 400px), slides from right 200ms ease, dark overlay
- Title "Project Settings"
- Sections: "General" (name, description), "Timeline" (start/due dates), "Team" (assigned team, project lead dropdown), "Status" (dropdown), "Visibility" (toggle public/private)
- Opens from gear icon in ProjectHeader. Closes on outside click or Escape.
- Fields pre-fill with current values. "Save Changes" button updates project row in DB. Show green toast "Settings saved". Close panel.
- On error: Red toast "Failed to save settings."

4. PROJECT HEADER (ProjectHeader.tsx):
- Horizontal bar at top of project view
- Left: Project name (bold, 20px), status badge (clickable — opens dropdown: "Active", "Planning", "On Hold", "Completed" — selecting updates status in DB), priority indicator
- Right: Team avatar stack (overlapping circles), due date with calendar icon, settings gear icon (opens ProjectSettings), three-dot menu, "Create Task" button (opens CreateTaskModal)

5. PROJECT SIDEBAR LINKS:
- Below existing team list in TeamList.tsx, add section header "Projects" with "+" icon
- On mount: Load projects from DB. Display each with name and colored dot for status.
- Clicking "+" opens CreateProjectModal. Clicking a project navigates to it.

6. DATABASE — PROJECTS TABLE:
- Create table "projects": id (UUID, PK), workspace_id (UUID, FK to workspaces), name (text, not null), description (text), team_id (UUID, FK to teams, nullable), status (text, default "Planning"), priority (text, default "Medium"), start_date (date, nullable), due_date (date, nullable), owner_id (UUID, FK to users), template (text, nullable), visibility (text, default "public"), progress (integer, default 0), created_at (timestamp, default now), updated_at (timestamp, default now)

=== PART B: TASKS ===

7. TASK LIST (TaskList.tsx):
- Vertical list of task rows inside a project view
- On mount: Load all tasks from tasks table where project_id matches, order by sort_order.

- Each row: Checkbox (checking sets status "Completed" and updates DB), task title, priority colored dot (Urgent=red #DC2626, High=orange #F59E0B, Medium=yellow #EAB308, Low=green #22C55E — 8px circles), assignee avatar (small circle), due date text, status badge (Not Started=gray, In Progress=blue, In Review=purple, Blocked=red, Completed=green)
- "Add task" row at bottom: text input placeholder "Add a task..." with "+" icon. Typing and pressing Enter creates task in DB with title, status "Not Started", priority "Medium", defaults. Appears in list. Input clears.
- Clicking any task row opens TaskDetailPanel.
- Task rows: Height 44px, hover highlight, border-bottom 1px #E2E8F0
- Empty state: "No tasks yet. Add your first task below."

8. TASK DETAIL PANEL (TaskDetailPanel.tsx):
- Right slide-out panel (width 520px, white background, 250ms ease)
- Header: Task title (editable — click turns to input, Enter or blur saves to DB), "X" close icon. Clicking outside or "X" closes panel.
- Description textarea (placeholder "Add a description..." — click activates, blur saves to DB)
- Property rows (each row: label 140px gray text on left, value on right, height 40px, hover highlight):

  Row 1: "Status" — clicking opens dropdown: "Not Started", "In Progress", "In Review", "Blocked", "Completed". Selecting updates DB.
  Row 2: "Assignee" — clicking opens dropdown of all members from workspace_members. Selecting shows name + avatar. "Remove assignee" option resets to "Unassigned". Saves to DB.
  Row 3: "Priority" — clicking opens dropdown: "Urgent" (red), "High" (orange), "Medium" (yellow), "Low" (green). Saves to DB.
  Row 4: "Due Date" — clicking opens date picker. Display as "Mar 15, 2026". Clear option. Saves to DB.
  Row 5: "Start Date" — same as Due Date.
  Row 6: "Estimated Time" — clicking opens number input + unit selector (hours/minutes). Display as "2h 30m". Saves to DB as total minutes.
  Row 7: "Actual Time" — same input pattern. Display as "1h 45m". Saves to DB.
  Row 8: "Tags" — clicking opens text input. Type tag + Enter adds colored pill. Each pill has "X" to remove. Stored as jsonb array in DB.
  Row 9: "Dependencies" — clicking opens dropdown listing all other tasks in project. Multi-select. Show as "Blocked by: [task names]". Saves to task_dependencies table.

- Below properties: "Subtasks" section (see SubtaskList below)
- Below subtasks: "Activity" section with header "Activity will appear here"

9. CREATE TASK MODAL (CreateTaskModal.tsx):
- Modal (max-width 520px), Title "Create Task"

- Fields: Task Name (placeholder "What needs to be done?" — required), Description textarea, Assignee dropdown (from workspace_members), Priority dropdown, Due Date picker, Estimated Time input (number + h/m unit), Tags input
- Buttons: "Cancel" (outline), "Create Task" (solid blue)
- Opens from "Create Task" button in ProjectHeader. Standard modal close behavior.
- On valid submit: Insert into tasks table. Show in task list. Close modal. Toast.

10. SUBTASK LIST (SubtaskList.tsx):
- Section inside TaskDetailPanel under "Subtasks" header
- Progress text: "2 of 4 complete" (auto-calculated from DB)
- Each subtask row: Checkbox (checking marks complete, strikethrough text, updates DB), subtask text, "X" delete icon on hover (deletes from DB)
- "Add subtask" input at bottom (placeholder "Add a subtask..." — Enter adds to subtasks table)
- Empty state: "No subtasks. Add a subtask below."

11. DATABASE — TASKS TABLE:
- Create table "tasks": id (UUID, PK), project_id (UUID, FK to projects), title (text, not null), description (text), status (text, default "Not Started"), priority (text, default "Medium"), assignee_id (UUID, FK to users, nullable), start_date (date, nullable), due_date (date, nullable), estimated_minutes (integer, nullable), actual_minutes (integer, default 0), tags (jsonb, default []), sort_order (integer, default 0), created_by (UUID, FK to users), created_at (timestamp, default now), updated_at (timestamp, default now), completed_at (timestamp, nullable)

12. DATABASE — SUBTASKS TABLE:
- Create table "subtasks": id (UUID, PK), task_id (UUID, FK to tasks), title (text, not null), is_completed (boolean, default false), sort_order (integer, default 0), created_at (timestamp, default now)

13. DATABASE — TASK_DEPENDENCIES TABLE:
- Create table "task_dependencies": id (UUID, PK), task_id (UUID, FK to tasks), depends_on_task_id (UUID, FK to tasks), created_at (timestamp, default now)
- Unique constraint on (task_id, depends_on_task_id)

=== DESIGN REQUIREMENTS ===
- Status badges: Rounded pill, 12px font, colors as specified above
- Slide-out panels: 200ms ease, dark overlay behind
- Priority dots: 8px circles with specified colors
- All inputs: 40px height, 6px border-radius, #CBD5E1 border
- Toasts: Top-right, green success / red error, 3s auto-dismiss

=== BOUNDARIES — CRITICAL ===
- Build everything described: UI + interactions + database in one pass
- Do not modify any files or tables from Prompt 1 except adding the Projects sidebar section to TeamList.tsx

- Do not add features from future prompts

PROMPT 3 OF 6 — TASK VIEWS & TEAM COLLABORATION

🔵 REPLIT SETTINGS: AUTONOMOUS MODE → ECONOMY | App Testing: OFF | Code Optimization: OFF

(Copy everything below this line into Replit)

CONTEXT:
App: AI-driven team and workplace productivity platform.
Feature being built: 6 task view layouts (Board, List, Timeline, Calendar, Table, Dashboard) and team collaboration (messaging, comments, documents, wiki).
Current state: Workspaces, Teams, Members, Projects, Tasks all working from Prompts 1-2.
Goal: Build all task views and collaboration tools — UI, interactions, database — in one pass.

Files to create:
src/components/views/ViewSwitcher.tsx
src/components/views/BoardView.tsx
src/components/views/ListView.tsx
src/components/views/TimelineView.tsx
src/components/views/CalendarView.tsx
src/components/views/TableView.tsx
src/components/views/DashboardView.tsx
src/pages/Messages.tsx
src/components/collab/ChannelList.tsx
src/components/collab/ChatWindow.tsx
src/components/collab/TaskComments.tsx
src/components/collab/DocumentEditor.tsx
src/components/collab/WikiPage.tsx

TASK:
Build 6 different task view layouts that all read from the same tasks database, plus a complete team collaboration system with messaging, task comments, a document editor, and wiki — all with full UI, interactions, and database.

IMPLEMENTATION DETAILS:

=== PART A: TASK VIEWS ===

1. VIEW SWITCHER (ViewSwitcher.tsx):
- Horizontal tab bar below project header
- 6 tabs with icons: Board (columns icon), List (list icon), Timeline (horizontal bar icon), Calendar (calendar icon), Table (grid icon), Dashboard (chart icon)
- Clicking switches active view (blue underline + blue text). Only one visible at a time.
- Store selected view in React state. Default to Board.
- DATABASE: Add column "preferred_view" (text, default "board") to workspace_members table. On view switch: save to DB. On page load: load and open saved view.

2. BOARD VIEW — KANBAN (BoardView.tsx):
- Horizontal columns: "Not Started", "In Progress", "In Review", "Completed"
- Each column: Header with name + task count, vertical stack of task cards
- Task card: White card (rounded 8px, shadow, padding 12px, gap 8px), title, priority dot, assignee avatar, due date
- Cards populated from tasks DB grouped by status
- DRAG AND DROP: User drags card between columns. Dropping changes task status in DB immediately.
- Clicking a card opens TaskDetailPanel from Prompt 2.
- "Add task" text at bottom of each column: inline input, Enter creates task with that column's status.

3. LIST VIEW (ListView.tsx):
- Grouped vertical list by status: "Not Started (3)", "In Progress (2)", etc.
- Group headers clickable to collapse/expand (200ms animation). Expanded by default.
- Task rows same as TaskList. Clicking opens TaskDetailPanel.

4. TIMELINE VIEW — GANTT (TimelineView.tsx):
- Left column: Task names (200px width)
- Right: Horizontal time grid with dates across top, 4 weeks total, horizontal scroll
- Each task: Colored horizontal bar (height 28px, rounded 4px, colored by status) showing start-to-end date span
- Tasks without dates: gray placeholder bar at today's position
- Today marker: Vertical red dashed line
- Clicking a bar opens TaskDetailPanel.

5. CALENDAR VIEW (CalendarView.tsx):
- Monthly grid (7 columns for days, 5-6 rows)
- Header: Month name + left/right arrows for navigation
- Day cells: Date number top-left, task pills stacked (height 20px, font 11px, rounded 3px, colored by status, name truncated)
- Today cell: Blue border highlight
- Tasks placed on their due_date. Tasks without due dates not shown.
- Clicking a pill opens TaskDetailPanel.

6. TABLE VIEW (TableView.tsx):
- Full spreadsheet-style table. Columns: Checkbox, Task Name, Status, Priority, Assignee, Due Date, Estimated Time, Tags, Progress
- Cells: Height 40px, border-bottom 1px #E2E8F0
- Column headers clickable for sort (toggle asc/desc, arrow icon)
- Checkbox: Checking marks task completed in DB
- Clicking task name opens TaskDetailPanel.

7. DASHBOARD VIEW (DashboardView.tsx):
- Grid of 4 summary cards (2x2 desktop, white, rounded 8px, shadow, padding 20px):
  Card 1: "Tasks by Status" — Pie chart (calculate counts per status, render colored segments with labels using CSS/SVG)
  Card 2: "Tasks by Priority" — Horizontal bar chart (count per priority, render bars with labels and counts)
  Card 3: "Upcoming Deadlines" — List of next 5 tasks by due date with days remaining
  Card 4: "Team Workload" — Horizontal bars per assignee showing their task count

=== PART B: TEAM COLLABORATION ===

8. MESSAGES PAGE (Messages.tsx):
- Two-column layout: Channel list on left (240px), chat window on right
- Page title "Messages" at top

9. CHANNEL LIST (ChannelList.tsx):
- Sections: "Project Channels" (auto-created when projects are created — "#" prefix + project name), "Direct Messages" (avatar + name), "Team Channels" ("#" prefix)
- "New Channel" link at bottom
- Clicking highlights channel (blue background), loads its messages in ChatWindow
- Load channels from DB on mount. Rows: 14px font, 36px height, hover highlight.

10. CHAT WINDOW (ChatWindow.tsx):
- Header: Channel name, member count, search icon, settings icon
- Scrollable message area. Each message: Avatar circle (40px), name (bold), timestamp, message text.
- Input at bottom: Text input (placeholder "Type a message...", height 44px, rounded 22px, border 1px #CBD5E1), attachment icon, emoji icon, send button (blue arrow)
- Type message, click send or press Enter. Message saved to messages table. Appears in chat with current user info and timestamp. Input clears.
- On channel open: Load messages from DB for that channel.
- Empty: "No messages yet. Start the conversation."

11. TASK COMMENTS (TaskComments.tsx):
- Section inside TaskDetailPanel below Activity header (replaces placeholder)

- Comment input at bottom: Avatar + text input (placeholder "Write a comment..."). Enter or click sends.
- On post: Insert into task_comments table. Refresh list. Comment count updates.
- Each comment: Avatar, name, timestamp, text, reply link. Load from DB with task detail.
- Empty: "No comments yet. Be the first to comment."

12. DOCUMENT EDITOR (DocumentEditor.tsx):
- Full page with toolbar at top: Bold, Italic, Underline, Heading, List, Code, Link, Image buttons (32px square, rounded 4px, gray icon, hover highlight)
- Title input (placeholder "Untitled Document", large font)
- Content area (placeholder "Start writing...")
- Right sidebar (240px): "Page Info" — Created by, Created date, Last edited, Word count
- Toolbar toggles formatting on selected text. Title and content editable. Saves to documents table (debounced 2 seconds).

13. WIKI PAGE (WikiPage.tsx):
- Two-column: Sidebar (240px) with page tree, content area (same as DocumentEditor)
- Sidebar: "Wiki" header, tree of pages (expandable with sub-pages). Clicking expand arrow shows/hides sub-pages (200ms).
- Clicking a page loads its content from DB.
- "New Page" button at sidebar bottom: Opens modal for page title, creates empty page (documents table with is_wiki=true).
- Empty: "No pages created. Add your first wiki page."

14. DATABASE — CHANNELS TABLE:
- Create table "channels": id (UUID, PK), workspace_id (UUID, FK), name (text), type (text: "project"/"direct"/"team"), project_id (UUID, FK, nullable), created_at (timestamp)

15. DATABASE — MESSAGES TABLE:
- Create table "messages": id (UUID, PK), channel_id (UUID, FK to channels), sender_id (UUID, FK to users), content (text), created_at (timestamp)

16. DATABASE — TASK_COMMENTS TABLE:
- Create table "task_comments": id (UUID, PK), task_id (UUID, FK to tasks), author_id (UUID, FK to users), content (text), parent_comment_id (UUID, nullable), created_at (timestamp)

17. DATABASE — DOCUMENTS TABLE:
- Create table "documents": id (UUID, PK), workspace_id (UUID, FK), title (text), content (text), created_by (UUID, FK), is_wiki (boolean, default false), parent_document_id (UUID, nullable), sort_order (integer, default 0), created_at (timestamp), updated_at (timestamp)

18. AUTO-CREATE CHANNELS: When a project is created (from Prompt 2), also create a project channel automatically in channels table.

=== BOUNDARIES — CRITICAL ===
- Build everything described: all 6 views + all collaboration tools + database
- All views read from the existing tasks table. Board drag-drop writes status changes.
- Do not modify any files or tables from Prompts 1-2 except: adding preferred_view column to workspace_members, adding TaskComments into TaskDetailPanel, and auto-creating channels on project creation
- Do not add features from future prompts

PROMPT 4 OF 6 — TIME TRACKING & AI PRODUCTIVITY ENGINE

🔵 REPLIT SETTINGS: AUTONOMOUS MODE → ECONOMY | App Testing: OFF | Code Optimization: OFF

(Copy everything below this line into Replit)

CONTEXT:
App: AI-driven team and workplace productivity platform.
Feature being built: Time tracking (timer, timesheets, reports) and AI Productivity Engine (scores, comparisons, profiles, team health, performer identification).
Current state: Workspaces, Teams, Members, Projects, Tasks, Views, Collaboration all working from Prompts 1-3.
Goal: Build complete time tracking and productivity analytics — UI, interactions, calculations, database — in one pass.

Files to create:
src/components/time/TimeTracker.tsx
src/components/time/TimesheetPage.tsx
src/components/time/TimeReport.tsx
src/pages/ProductivityDashboard.tsx
src/components/productivity/ProductivityScore.tsx
src/components/productivity/ComparisonCards.tsx
src/components/productivity/PerformanceProfile.tsx
src/components/productivity/TeamHealthDashboard.tsx
src/components/productivity/PerformerIdentification.tsx
src/utils/productivityCalculator.ts

TASK:

Build the complete time tracking system and the full AI productivity engine with scoring algorithms, comparisons, performance profiles, team health, and performer identification — UI, logic, calculations, and database.

IMPLEMENTATION DETAILS:

=== PART A: TIME TRACKING ===

1. TIME TRACKER WIDGET (TimeTracker.tsx):
- Small floating widget, fixed position bottom-right (20px from edges, z-index above content)
- Collapsed state: Small clock icon (36px circle) — click to expand
- Expanded: Current task name (truncated 30 chars), timer "00:00:00" (monospace, 24px, bold), Play button (green #22C55E bg, white icon, circle 36px), Pause button (blue), Stop button (red, square 28px)
- "Today: 0h 0m" summary below (sum all time_entries for today)
- Task name clickable — opens dropdown of "In Progress" tasks to switch
- Play: Start counting, update every second. When task is "In Progress", auto-start.
- Pause: Freeze timer, button changes to Play to resume.
- Stop: Stop timer. Insert time_entry into DB (task_id, date=today, minutes=elapsed, source="timer"). Add elapsed to task actual_minutes in tasks table. Reset to 00:00:00. Show minimize icon to collapse.
- On error: Red toast.

2. TIMESHEET PAGE (TimesheetPage.tsx):
- Page title "Timesheets" with week selector (left arrow, "Mar 3 - Mar 9, 2026", right arrow)
- Table: Rows = task names, Columns = Mon/Tue/Wed/Thu/Fri/Sat/Sun + Total
- Each cell: Time display "0:00". Clicking turns into time input. Accept "1:30" or "1.5" (both = 1h 30m). Enter or blur saves.
- On cell edit: Upsert time_entry for user+task+date. If exists update minutes, else insert with source="manual".
- Row totals: Auto-sum all day cells. Column totals: Auto-sum all task cells. Bottom row bold totals.
- "Submit Timesheet" button bottom-right. Status badge: "Draft" (gray), "Submitted" (green), "Approved" (blue).
- Submit: Insert/update timesheets row status="submitted". Disable editing when Submitted or Approved.
- Week navigation loads data from DB for selected week.
- Timesheet cells: 60px width, centered text

3. TIME REPORT (TimeReport.tsx):
- Header "Time Report" with date range selector: "This Week", "Last Week", "This Month", "Last Month", "Custom Range"
- Summary cards row: "Total Hours" (large number), "Avg Hours/Day", "Most Time Spent On" (project name), "Overtime Hours"

- Horizontal bar chart: Hours per project (CSS width percentages)
- Table: Date, Task, Project, Time, Notes
- All calculated from time_entries table filtered by date range.
- Empty: "No time entries for this period"

4. DATABASE — TIME_ENTRIES TABLE:
- Create table "time_entries": id (UUID, PK), user_id (UUID, FK), task_id (UUID, FK), project_id (UUID, FK), date (date), minutes (integer), notes (text, nullable), created_at (timestamp), source (text: "timer"/"manual")

5. DATABASE — TIMESHEETS TABLE:
- Create table "timesheets": id (UUID, PK), user_id (UUID, FK), workspace_id (UUID, FK), week_start (date), status (text, default "draft"), submitted_at (timestamp, nullable), approved_by (UUID, nullable), approved_at (timestamp, nullable)

=== PART B: AI PRODUCTIVITY ENGINE ===

6. PRODUCTIVITY DASHBOARD (ProductivityDashboard.tsx):
- Page title "Productivity Intelligence" with subtitle "AI-powered team analytics"
- Tabs: "Overview", "Individual Profiles", "Team Health", "Comparisons"
- Tab switching: Active = blue underline + blue text. Content area switches.
- "Individual Profiles" tab: Dropdown to select employee, loads their profile.

7. PRODUCTIVITY SCORE (ProductivityScore.tsx):
- Large card with circle progress ring (120px diameter, 8px stroke, stroke-dasharray animation) — score 0-100 in center
- Label: Performance level text
- 7 factor horizontal bars below (height 8px, rounded, background #E2E8F0, fill colored by range: 90-100=green, 75-89=blue, 60-74=yellow, 40-59=orange, below 40=red):
  "Task Completion Rate", "Quality Score", "Estimation Accuracy", "Collaboration Score", "Initiative Score", "Consistency", "Impact Weight"
- Values calculated by productivityCalculator (see #10)

8. COMPARISON CARDS (ComparisonCards.tsx):
- Row of 5 cards (180px width each, white, shadow, responsive wrap):
  "Today vs Yesterday" — task counts, percentage change
  "This Week vs Last Week" — task counts
  "This Month vs Last Month" — task counts
  "Avg Completion Time" — current vs previous (lower = green)
  "Hours Worked" — current vs previous
- Green arrow up for positive changes. Red arrow down for negative.
- Calculated from tasks and time_entries via productivityCalculator.

9. PERFORMANCE PROFILE (PerformanceProfile.tsx):

- Header: Employee name, role, team, avatar (64px)
- Score section: ProductivityScore component
- Trend section: 30-day line chart (plot daily score as points connected by line, CSS/SVG positioning by % of container width). Data from productivity_snapshots.
- "AI-Identified Strengths" section: 3 items generated from highest score factors
- "Growth Opportunities" section: 2 items generated from lowest score factors
- Work output: "Tasks Completed", "Hours Logged", "Projects" — from DB
- Activity heatmap: 7x24 grid (15px cells, white to dark blue intensity by activity level)

10. PRODUCTIVITY CALCULATOR (productivityCalculator.ts):
- calculateProductivityScore(userId, dateRange):
  Task Completion Rate (25% weight): (completed / assigned) * 100
  Quality Score (20%): 100 - (tasks_sent_back / completed * 100). Default 80 if no data.
  Estimation Accuracy (10%): 100 - abs(estimated - actual) / estimated * 100. Cap 0 min. Default 70.
  Collaboration Score (15%): Comments on others' tasks (40%) + response time inverted (30%) + unblocking tasks (30%). Normalize 0-100.
  Initiative Score (10%): Unassigned tasks picked up / total * 100
  Consistency (10%): Inverted std deviation of daily completions. Normalize 0-100.
  Impact Weight (10%): Urgent=4x, High=3x, Medium=2x, Low=1x. Normalize vs team avg.
  Final = weighted sum, rounded integer.
  Labels: 90-100="Exceptional", 75-89="High Performer", 60-74="Steady Contributor", 40-59="Needs Attention", 0-39="At Risk"

- calculateComparisons(entityId, entityType):
  Count completed tasks current vs previous period. Percentage change.
  Periods: today/yesterday, thisWeek/lastWeek, thisMonth/lastMonth, thisQuarter/lastQuarter.
  Also: avg completion time, total hours.
  Direction: positive=green up, negative=red down.

- identifyPerformers(teamId):
  Score all team members. Sort descending. Top 3 = Top Performers. Bottom 3 with score<60 = Needs Attention.
  Generate recommendation text from strongest/weakest factors.

11. TEAM HEALTH DASHBOARD (TeamHealthDashboard.tsx):
- Grid of metric cards (2 columns):
  "Team Velocity": Line chart — tasks/week for last 6 weeks (CSS/SVG)
  "Bottleneck Detection": Text showing blocked tasks count + reason
  "Workload Balance": Horizontal bars per member with capacity % (Green 0-79%, Orange 80-100%, Red >100%)
  "Burnout Risk": Employee rows with risk badge (Low=green, Medium=orange, High=red) — based on sustained overtime
  "Team Morale Proxy": Gauge 0-100 with label

"Sprint Velocity Trend": Bar chart last 6 sprints

12. PERFORMER IDENTIFICATION (PerformerIdentification.tsx):
- Two sections side by side:
  Left: "Top Performers" (gold star icon) — 3 cards with rank (#1,#2,#3), name, score, trend arrow, AI recommendation (gold border)
  Right: "Needs Attention" (alert icon) — 3 cards with name, score (orange/red), trend arrow down, AI suggestion (orange border)
- Privacy notice: "Visible to direct managers and above only"
- When no data: "Productivity data will appear after one week of activity"

13. DATABASE — PRODUCTIVITY_SNAPSHOTS TABLE:
- Create table "productivity_snapshots": id (UUID, PK), user_id (UUID, FK), workspace_id (UUID, FK), date (date), overall_score (integer), task_completion_rate (integer), quality_score (integer), estimation_accuracy (integer), collaboration_score (integer), initiative_score (integer), consistency_score (integer), impact_weight (integer), tasks_completed (integer), hours_worked (integer), created_at (timestamp)
- Unique constraint on (user_id, date)
- On first dashboard load each day: Calculate score for current user, insert if no row for today.
- Load last 30 days for trend chart. Last 90 for profile.
- Comparison calculations query tasks and time_entries directly with date filters.
- Team queries load all member snapshots.

=== BOUNDARIES — CRITICAL ===
- Build everything described: time tracking + full productivity engine + database
- Do not modify files or tables from Prompts 1-3 except updating task actual_minutes on timer stop
- Do not add features from future prompts

PROMPT 5 OF 6 — WORKLOAD MANAGEMENT & REPORTING SUITE

🔵 **REPLIT SETTINGS: AUTONOMOUS MODE → ECONOMY | App Testing: OFF | Code Optimization: OFF**

(Copy everything below this line into Replit)

CONTEXT:
App: AI-driven team and workplace productivity platform.

Feature being built: Workload & resource management + full reporting suite with executive dashboard, custom report builder, and report library.
Current state: Workspaces, Teams, Members, Projects, Tasks, Views, Collaboration, Time Tracking, Productivity Engine all working from Prompts 1-4.
Goal: Build workload management and complete reporting suite — UI, interactions, calculations, database — in one pass.

Files to create:
src/pages/Workload.tsx
src/components/workload/WorkloadView.tsx
src/components/workload/CapacityPlanner.tsx
src/pages/Reports.tsx
src/components/reports/ExecutiveDashboard.tsx
src/components/reports/ReportBuilder.tsx
src/components/reports/ReportLibrary.tsx

TASK:
Build complete workload management with capacity planning and drag-drop rebalancing, plus the full reporting suite with executive dashboard, custom report builder with filters/grouping/charts, and a report template library — UI, interactions, calculations, and database.

IMPLEMENTATION DETAILS:

=== PART A: WORKLOAD & RESOURCES ===

1. WORKLOAD PAGE (Workload.tsx):
- Page title "Workload & Resources" with team selector dropdown (lists all teams from DB, selecting filters members)
- Two tabs: "Current Workload", "Capacity Planning". Click to switch. Active = blue underline.
- No team selected: "Select a team to view workload"

2. WORKLOAD VIEW (WorkloadView.tsx):
- Vertical list of team members
- For each member: Sum estimated_minutes of all assigned incomplete tasks from DB. Capacity = 40h/week (from weekly_capacity_hours column). Utilization = assigned hours / capacity * 100.
- Each row: Avatar, Name, Role, Capacity bar (horizontal, height 24px, rounded 4px, animated fill on load)
  Green = 0-79%, Orange = 80-100%, Red = over 100%
- Percentage text on right: "65%", "95%", "120%"
- Below bar: Small task pill tags showing assigned task names
- DRAG-DROP REBALANCE: Task pills draggable from one member to another. Dropping updates assignee_id in tasks table. Capacity bars recalculate immediately.
- No tasks: "No tasks assigned. Team is fully available."

3. CAPACITY PLANNER (CapacityPlanner.tsx):
- Horizontal timeline (4 weeks)
- Left column: Member names (200px)
- Grid cells (40px square, border 1px #E2E8F0): Colored blocks showing planned hours/day
  Green = available, Orange = near capacity, Red = overloaded, Gray striped = PTO/holiday
- Calculate hours per day: Distribute task estimated_minutes evenly across work days between start_date and due_date.
- Summary row: "Team Available Hours: Xh" and "Team Committed: Xh"
- "Add PTO" button per member: Opens small modal (date picker, type selector: PTO/Holiday/Sick, notes). Saves to member_availability table. Gray striped block appears.

4. DATABASE — MEMBER_AVAILABILITY TABLE:
- Create table "member_availability": id (UUID, PK), user_id (UUID, FK), date (date), type (text: "pto"/"holiday"/"sick"), notes (text, nullable)

5. DATABASE — ADD COLUMN:
- Add "weekly_capacity_hours" (integer, default 40) to workspace_members table.

=== PART B: REPORTING SUITE ===

6. REPORTS PAGE (Reports.tsx):
- Title "Reports & Analytics"
- Tabs: "Executive Dashboard", "Custom Reports", "Report Library". Tab switching with blue underline.

7. EXECUTIVE DASHBOARD (ExecutiveDashboard.tsx):
- Grid of 6 summary cards (3x2, white, shadow, 200px min-height, centered content, large numbers 36px bold, trend text 14px):
  "Company Productivity Score" — Average of all member productivity scores from productivity_snapshots + trend arrow
  "Active Projects" — Count from projects where status="Active" + "[n] at risk" in red (>30% tasks overdue)
  "Tasks Completed This Week" — Count from tasks where completed_at in current week + % change from last week
  "Team Utilization" — Average capacity % from workload calculations
  "On-Time Delivery Rate" — (Tasks completed on/before due_date / total completed) * 100 + trend line
  "Average Task Completion Time" — Average actual_minutes from completed tasks this month + comparison
- Below: "Department Performance Comparison" bar chart (CSS/SVG)
- Below: "Projects at Risk" table: Project, Risk Level, Issue, Owner — projects with >30% overdue tasks or declining velocity

8. REPORT BUILDER (ReportBuilder.tsx):
- Left panel (240px): Data source selector. Sections: "Tasks", "Time", "Members", "Projects" with checkboxes for fields. Checking adds columns to canvas.
- Center: Report canvas. Default: "Drag fields here to build report". After generation: Renders chart/table.
- Top toolbar: Chart type icons (bar, line, pie, table) — clicking changes visualization. "Add Filter" button (opens row: field dropdown, operator: equals/contains/greater than/less than/between, value input). "Group By" dropdown (options based on selected source: by team, project, week, status, priority).
- Right panel (200px): Formatting options placeholder.
- "Generate Report": Validate trigger+fields selected. Process fields, apply filters and grouping, render as selected chart type from DB data.
- "Export" dropdown (PDF, Excel, CSV) — show toast "Export coming soon" for now.
- "Save Report" button (appears after generation): Save config as JSON to saved_reports table. Show in "My Reports" section.
- Canvas empty: "Select data fields and click Generate to create your report"

9. REPORT LIBRARY (ReportLibrary.tsx):
- Grid of report template cards (3 columns, white, shadow, 180px height, hover scale(1.02))
- 10 cards:
  "Daily Standup Summary" — Group by member: tasks done yesterday, planned today, blockers
  "Weekly Productivity Report" — Group by team: tasks completed, hours, score, % change
  "Sprint Retrospective" — Group by sprint: planned, completed, carryover, velocity, estimation accuracy
  "Individual Performance Review" — Filter by member: all score factors, tasks, quality, growth areas
  "Project Health Report"
  "Workload Distribution"
  "Time Utilization Report"
  "Client Project Profitability"
  "Department Comparison"
  "Burnout Risk Report"
- Each card: Icon, title, 1-line description, "Generate" button
- Clicking "Generate": Pre-configure report builder with template's fields/grouping/chart, then auto-generate.
- No saved reports: "You haven't created any custom reports yet"

10. DATABASE — SAVED_REPORTS TABLE:
- Create table "saved_reports": id (UUID, PK), workspace_id (UUID, FK), name (text), configuration (jsonb — fields, filters, grouping, chart type), created_by (UUID, FK), created_at (timestamp), updated_at (timestamp), schedule (text, nullable: "daily"/"weekly"/"monthly"/null)
- Load saved reports in "Custom Reports" tab. Clicking one loads config and regenerates.
- Executive dashboard calculations query directly from DB for accuracy.

=== BOUNDARIES — CRITICAL ===
- Build everything described: workload + full reporting + database
- Do not modify files from Prompts 1-4 except adding weekly_capacity_hours column to workspace_members
- Do not add features from future prompts

PROMPT 6 OF 6 — GOALS/OKRs & AUTOMATION/AI

🔵 **REPLIT SETTINGS: AUTONOMOUS MODE → ECONOMY | App Testing: OFF | Code Optimization: OFF**

(Copy everything below this line into Replit)

CONTEXT:
App: AI-driven team and workplace productivity platform.
Feature being built: Goals & OKR system + Workflow automations + AI assistants (command bar, coach, smart search) + Notification center.
Current state: All features from Prompts 1-5 working: Workspaces, Teams, Members, Projects, Tasks, Views, Collaboration, Time Tracking, Productivity Engine, Workload, Reports.
Goal: Build the final two modules — Goals/OKRs and Automation/AI — completing the entire platform.

Files to create:
src/pages/Goals.tsx
src/components/goals/OKRTree.tsx
src/components/goals/GoalCard.tsx
src/components/goals/CreateGoalModal.tsx
src/pages/Automations.tsx
src/components/automation/AutomationBuilder.tsx
src/components/ai/AICommandBar.tsx
src/components/ai/AICoach.tsx
src/components/ai/SmartSearch.tsx
src/components/notifications/NotificationCenter.tsx

TASK:
Build the complete Goals & OKR system with cascading objectives and the complete Automation & AI system with workflow builder, AI command bar, AI coach, smart search, and notification center — UI, interactions, logic, and database.

IMPLEMENTATION DETAILS:

=== PART A: GOALS & OKRs ===

1. GOALS PAGE (Goals.tsx):
- Title "Goals & OKRs" with period selector dropdown ("Q1 2026", "Q2 2026", etc.)
- Tabs: "Company Goals", "Team Goals", "My Goals". Click to switch. Filter view by tab.
- "Add Goal" button (solid blue) opens CreateGoalModal.
- Empty: "No goals set for this period. Start by creating a company objective."
- "My Goals" empty: "You don't have any personal goals yet."

2. OKR TREE (OKRTree.tsx):
- Cascading tree visualization. Connecting lines: thin gray vertical + horizontal, 1px.
  Level 1: Company Objective (large card, blue 4px left border)
  Level 2: Department Key Results (medium, indented 40px, connected)
  Level 3: Team Objectives (smaller, further indented)
  Level 4: Individual Goals (smallest, most indented)
- Load goals from DB by workspace and period. Build tree from parent_goal_id relationships.
- Clicking a level expands/collapses children (200ms animation).
- Clicking a goal card opens an edit side panel (same style as TaskDetailPanel — slide-out 520px, fields pre-filled, save updates DB).

3. GOAL CARD (GoalCard.tsx):
- Card (white, shadow, rounded 8px, left border 4px colored by level): Title (bold), description (1 line truncated), progress bar (height 6px, rounded, #E2E8F0 bg, colored by confidence), percentage text, owner avatar + name, due date
- Confidence badge: "On Track" (green), "At Risk" (orange), "Off Track" (red). Clicking opens dropdown to change. Selection updates DB.
- Progress: If goal has linked tasks (goal_task_links), auto-calculate (completed linked / total linked * 100). Manual goals: clicking progress bar opens number input. Saves to DB.

4. CREATE GOAL MODAL (CreateGoalModal.tsx):
- Modal (max-width 520px), Title "Create Goal"
- Fields: Goal Title (required), Description textarea, Goal Type dropdown ("Company Objective"/"Department Key Result"/"Team Goal"/"Individual Goal"), Parent Goal dropdown (shows goals from level above for alignment), Owner dropdown, Due Date picker, Target Value (number), Current Value (number), Measurement Unit ("percentage"/"count"/"currency"/"custom")
- Buttons: "Cancel", "Create Goal"
- On submit: Insert into goals table. Show in correct tab and OKR tree position. Toast.

5. DATABASE — GOALS TABLE:
- Create table "goals": id (UUID, PK), workspace_id (UUID, FK), title (text, not null), description (text), goal_type (text: "company"/"department"/"team"/"individual"), parent_goal_id (UUID, FK self-referencing, nullable), owner_id (UUID, FK), team_id (UUID, FK, nullable), period (text),

target_value (decimal, nullable), current_value (decimal, default 0), measurement_unit (text), confidence (text, default "on_track"), due_date (date, nullable), created_at (timestamp), updated_at (timestamp)

6. DATABASE — GOAL_TASK_LINKS TABLE:
- Create table "goal_task_links": id (UUID, PK), goal_id (UUID, FK to goals), task_id (UUID, FK to tasks)

=== PART B: WORKFLOW AUTOMATION ===

7. AUTOMATIONS PAGE (Automations.tsx):
- Title "Automations" with "Create Automation" button
- List of automation cards: Name, trigger description, action description, active toggle (ON=green, OFF=gray), edit icon
- Cards: White, shadow, 60px height, toggle on right
- Load from automations table. Toggle updates is_active in DB.
- Empty: "No automations created. Automate repetitive tasks to save time."

8. AUTOMATION BUILDER (AutomationBuilder.tsx):
- Visual flow: "WHEN" → "IF" → "THEN" with arrow connectors (vertical, pointing down)
- Name input at top (placeholder "Name this automation")
- "WHEN" trigger dropdown options: "Task status changes to [status]", "Task is assigned to [person]", "Task is overdue by [days]", "All subtasks completed", "New member added to project", "Project reaches [%] completion"
- "IF" condition dropdown (optional): "Priority is [priority]", "Assignee is [person]", "Project is [project]", "Tag contains [tag]"
- "THEN" action dropdown: "Change task status to [status]", "Assign task to [person]", "Send notification to [person/team]", "Create a new task", "Move task to [project]", "Add tag [tag]"
- Each selection shows sub-fields for variables in brackets.
- "Add another action" link below THEN (max 5 actions).
- "Save Automation": Validate trigger required + at least 1 action. Save to automations table (trigger_config, conditions, actions as jsonb). Appears in list with toggle ON.
- "Test" button at bottom.

9. AUTOMATION EXECUTION ENGINE:
- Function that checks automations on every task status change, assignment change, and completion event.
- When trigger matches: Check conditions. If pass: Execute all actions in order.
- Log every execution to automation_logs table. Update last_triggered_at and trigger_count.

=== PART C: AI FEATURES ===

10. AI COMMAND BAR (AICommandBar.tsx):
- Floating bar, top center, white, large shadow, rounded 12px, z-index 1000

- Opens on Ctrl+K or Cmd+K. Closes on Escape.
- Large input (50px height, placeholder "Ask AI anything or type a command...")
- 4 suggestion chips below: "Create a task", "Show my tasks", "Team productivity this week", "Generate a report". Clicking fills input.
- Process by keyword matching (no external AI API):
  "create task [name]" → Create task in current project
  "show my tasks" → Navigate to task list filtered by user
  "show [team] productivity" → Navigate to productivity dashboard for team
  "search [query]" → Open smart search with query
- Result text below input confirming action.

11. AI COACH PANEL (AICoach.tsx):
- Right sidebar (width 320px), collapsible. Header "AI Coach" with sparkle icon + collapse icon.
- Light gray background, 14px text, compact spacing.
- "Today's Plan": Top 3 tasks by priority and due date from DB.
- "Focus Recommendations": Check for meeting-free blocks, suggest for high-priority tasks.
- "Insights": Compare this week's task completion to last week, generate insight text.
- "Ask your coach..." input at bottom with send. Same keyword matching, responses phrased as coaching advice.

12. SMART SEARCH (SmartSearch.tsx):
- Full-width overlay (z-index 1000, dark semi-transparent bg, white search area). Opens from nav bar search icon. Closes on Escape or bg click.
- Large search input (placeholder "Search tasks, docs, people, decisions...")
- Category filter tabs: "All", "Tasks", "Documents", "People", "Messages"
- Search across: task titles, document titles, member names, message content from DB.
- Results grouped by category. Clicking navigates to item.
- No results: "No results found for your search"

13. NOTIFICATION CENTER (NotificationCenter.tsx):
- Dropdown from bell icon in top nav (width 360px, max-height 480px)
- Header "Notifications" with "Mark all read" link
- Items: Icon, title (bold), description, timestamp, read/unread dot
  Unread: Blue dot left, light blue bg. Read: No dot, white bg.
- Padding 12px, border-bottom 1px #E2E8F0, hover highlight
- Generate notifications from actual events: task assignments, completions, approaching deadlines (within 24h), mentions in comments. Save to notifications table.
- "Mark all read": Update all is_read=true for user. Bell icon shows unread count badge.
- Clicking notification: Mark read, navigate to related item.
- Footer: "View all notifications" link
- Empty: "You're all caught up"

14. DATABASE — AUTOMATIONS TABLE:

- Create table "automations": id (UUID, PK), workspace_id (UUID, FK), name (text, not null), trigger_type (text), trigger_config (jsonb), conditions (jsonb, default []), actions (jsonb), is_active (boolean, default true), created_by (UUID, FK), created_at (timestamp), last_triggered_at (timestamp, nullable), trigger_count (integer, default 0)

15. DATABASE — AUTOMATION_LOGS TABLE:
- Create table "automation_logs": id (UUID, PK), automation_id (UUID, FK), triggered_at (timestamp), trigger_event (jsonb), actions_executed (jsonb), status (text: "success"/"error"), error_message (text, nullable)

16. DATABASE — NOTIFICATIONS TABLE:
- Create table "notifications": id (UUID, PK), workspace_id (UUID, FK), user_id (UUID, FK), type (text), title (text), body (text), related_entity_type (text: "task"/"project"/"comment"/etc.), related_entity_id (UUID), is_read (boolean, default false), created_at (timestamp)
- Save notifications on events. Load by user_id, order by created_at desc, limit 50.

=== BOUNDARIES — CRITICAL ===
- Build everything described: Goals/OKRs + Automations + AI features + Notifications + database
- Do not modify files from Prompts 1-5 except adding event hooks in task status/assignment/completion handlers for automation engine and notification generation
- This is the final prompt — the platform should be fully complete after this

---

TESTING CHECKLIST

After each prompt, verify these before moving to the next:

After Prompt 1: Create a workspace. Create 3 teams. Invite 2 members. Change a role. Remove a member. Refresh page — all data persists.

After Prompt 2: Create 2 projects. Create 5 tasks in a project. Edit every task property. Add subtasks. Set dependencies. Refresh — all persists.

After Prompt 3: Switch between all 6 views. Drag a card in Board view. Send a message. Post a comment. Create a wiki page. Refresh — all persists.

After Prompt 4: Start timer, stop, check time entry saved. Edit timesheet. View productivity score. Check comparison cards. View team health. Refresh — all persists.

After Prompt 5: View workload bars. Drag a task between members. Add PTO. View executive dashboard numbers. Build a custom report. Generate from library template. Refresh — all persists.

After Prompt 6: Create a company goal with key results. Create an automation and trigger it. Use Ctrl+K command bar. Open AI coach. Search across all data. Check notifications. Refresh — all persists.

IF SOMETHING BREAKS

- Stop. Do not send the next prompt.
- Describe the exact issue to Claude: which component, what you clicked, what happened vs expected.
- Get a small fix prompt. Test the fix. Then continue.